챕터 5. Kubernetes 네트워킹

오픈이는 지금까지 학습한 내용을 바탕으로 테스트 서버를 Pod로 재구성했습니다. Pod가 종료되더라도 Deployment가 즉시 새 Pod를 실행해 주니 마음이 든든했습니다.

다음 날 아침, 동료가 찾아왔습니다.

동료 : "오픈 씨, 어제 테스트 서버에 올렸다는 서비스, 접속이 안 되는데 어떻게 들어가요?"

오픈이는 당황했습니다. 로그상으로는 정상 실행 중이었지만, 정작 브라우저로 접속해 본 적은 없었기 때문입니다. 서둘러 Pod 목록에서 IP를 확인해 브라우저에 입력해 보았지만, 페이지는 열리지 않았습니다.

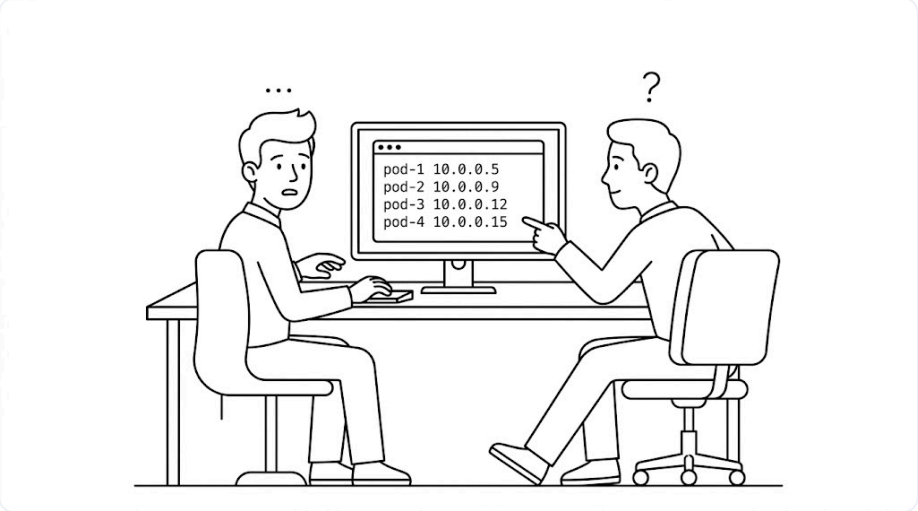


그림 5-1. Pod는 떠 있는데 접근할 주소가 없던 아침

게다가 Pod가 재생성될 때마다 IP가 바뀌니, IP로는 접근할 수도 없었습니다.

'쿠버네티스 환경에서는 외부에서 Pod로 어떻게 접속해야 하지?'

막막해진 오픈이가 선배에게 묻자, 선배가 화면을 보며 설명했습니다.

선배 : "외부 요청이 실제 Pod까지 도달하려면 두 가지가 필요해요. 들어온 요청을 도메인이나 경로에 따라 알맞은 곳으로 보내 주는 **Ingress**, 그리고 수시로 바뀌는 Pod를 묶어 고정된 진입점을 제공하는 **Service**죠. 이 두 가지를 한번 알아봐요."

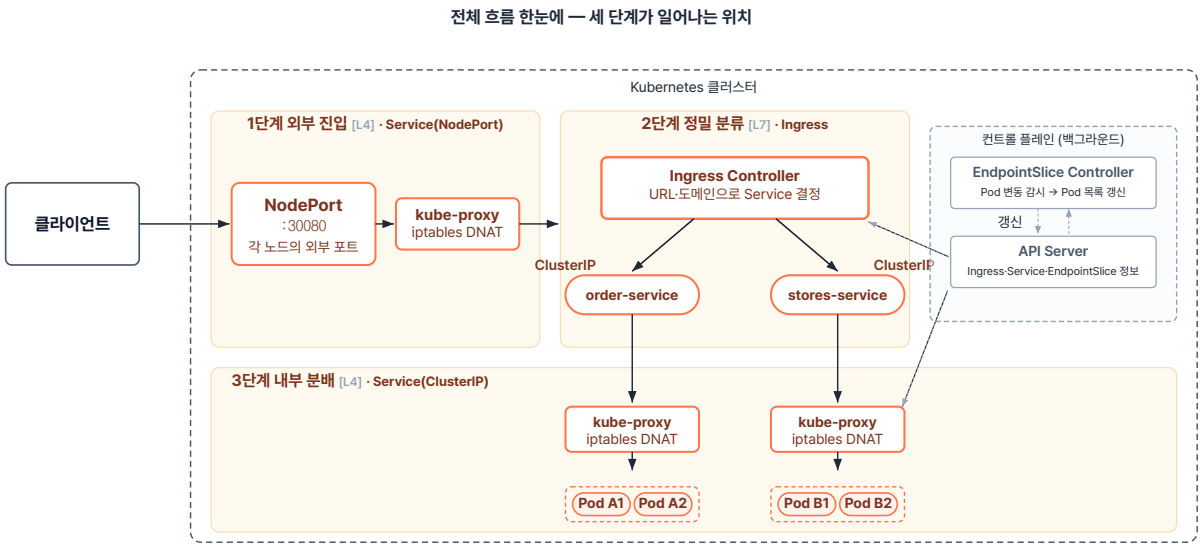


그림 5-2. 챕터 5 한눈에 보기 - 요청이 Pod까지 도달하는 전체 흐름

실습 준비. 예제 코드

이 책의 실습 코드는 GitHub 레포 하나에 챕터별 폴더로 담겨 있습니다. 아래 명령으로 레포를 한 번 git clone 합니다. 이후 챕터마다 해당 폴더로 이동해 실습합니다.

터미널 예제 코드 클론

```
git clone https://github.com/metacoding-10-linux-docker/start
```

이번 챕터는 `ex11` ~ `ex12` 폴더를 사용합니다.

완성된 코드는 아래 주소에서 확인하세요.

터미널 완성 코드

```
https://github.com/metacoding-10-linux-docker/final
```

5.1 Service - Pod의 고정 주소

5.1.1 Service가 필요한 이유

Pod의 IP는 고정된 값이 아닙니다. Pod는 종료되거나 새 버전으로 교체될 때마다 새로 만들어지고, 그때마다 IP가 바뀝니다. 게다가 같은 역할을 하는 Pod를 여러 개 띄워 두면, 요청을 그중 어디로 보낼지도 정해야 합니다.

Service는 이 문제를 해결합니다. Pod IP가 바뀌어도 변하지 않는 **고정 진입점**을 만들고, 들어온 요청을 **연결된 여러 Pod**에 **분배**하는 리소스입니다.

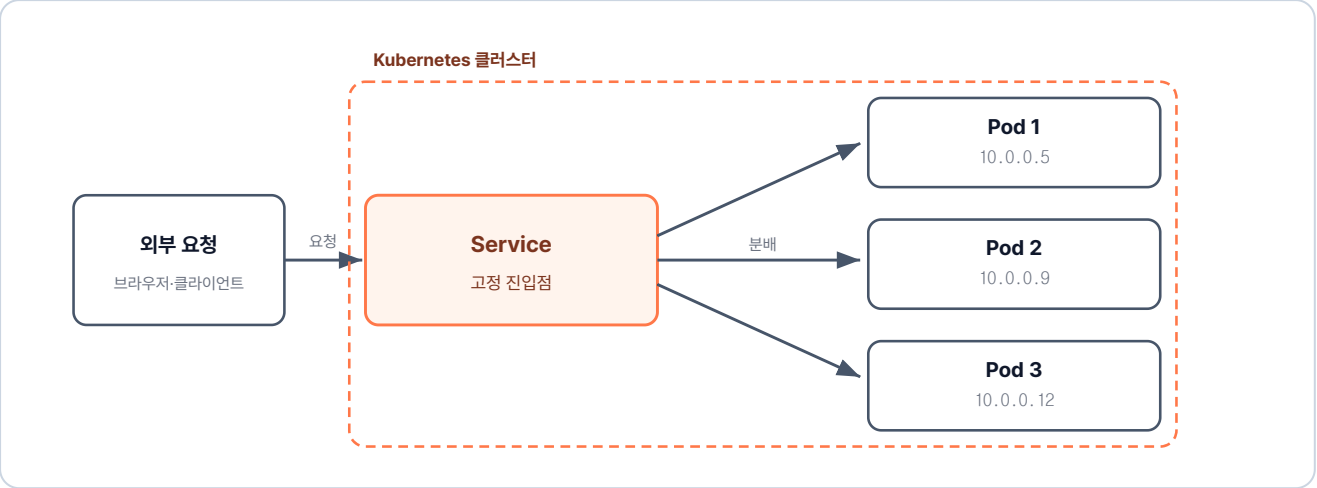


그림 5-3. Service는 Pod IP가 바뀌어도 변하지 않는 고정 주소를 제공

5.1.2 Service 생성

이제 Pod에 연결할 Service를 직접 만들어 보겠습니다.

실습 코드는 `ex11` 폴더에 있습니다. 이전 챕터에서 사용한 `deploy-ex02.yml`을 사용합니다.

```
ex11/
├── deploy-ex02.yml    # nginx Pod 4개를 유지하는 Deployment 정의
├── service-ex01.yml   # Pod 앞에 붙이는 NodePort Service 정의
└── README.md          # 실습 안내
```

다음은 Service를 정의한 YAML입니다.

실습 1 **ex11/service-ex01.yml. NodePort Service 정의**

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort        # 노드 IP와 포트로 외부 접근이 가능한 타입
  selector:
    app: nginx          # 이 라벨을 가진 Pod들을 뒤에 연결
  ports:
    - port: 80          # 클러스터 내부에서 Service로 진입하는 포트
      targetPort: 80    # Service가 Pod로 요청을 전달하는 포트
      nodePort: 30080   # 외부에서 노드로 진입 시 사용하는 포트 (기본 허용 범위 30000~32767)
```

Service는 **Deployment**와 마찬가지로 **selector**에 지정한 라벨과 일치하는 Pod를 골라 연결합니다.

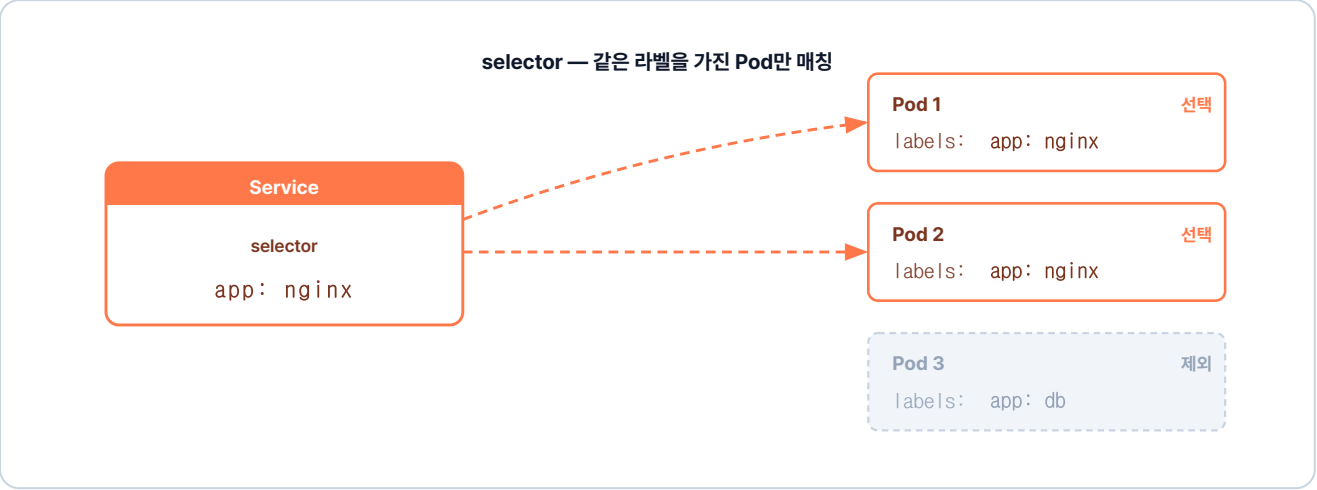


그림 5-4. selector가 지정한 라벨(app: nginx)을 가진 Pod만 매칭하고 다른 라벨은 제외

5.1.3 Service 타입과 접근 범위

앞서 작성한 Service YAML에는 `type: NodePort` 설정이 있습니다. Service는 접근 범위에 따라 타입이 **ClusterIP**, **NodePort**, **LoadBalancer**로 나뉩니다.

ClusterIP

ClusterIP는 사내에서만 연결되는 **내선 번호**입니다. **클러스터 안의 Pod끼리는 서로 연결되지만, 외부에서는 접근할 수 없습니다.** DB처럼 외부에 노출할 필요가 없는 구성에 적합하며, 서비스 타입의 기본값입니다.

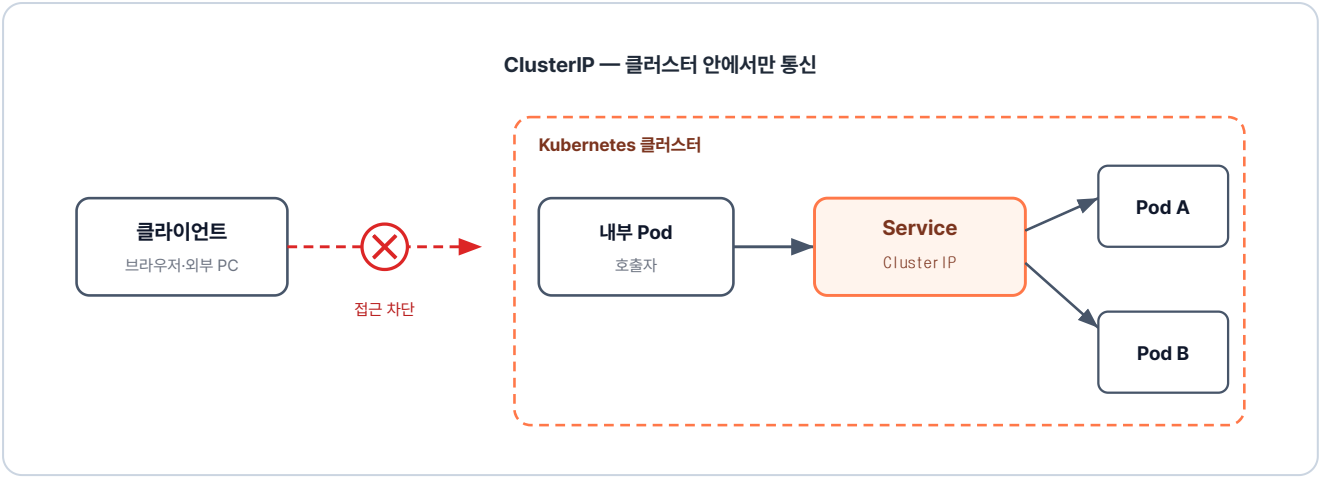


그림 5-5. ClusterIP는 외부 요청은 닿지 못하고 내부 Pod끼리만 통신

NodePort

NodePort는 외부에서도 연결할 수 있는 **직통 번호**입니다. Service를 NodePort로 지정하면, **노드(서버)에 외부와 통하는 포트가 열립니다.** 외부에서 그 노드의 IP와 포트로 접속하면, 요청이 클러스터 안의 Service를 거쳐 Pod로 전달됩니다.

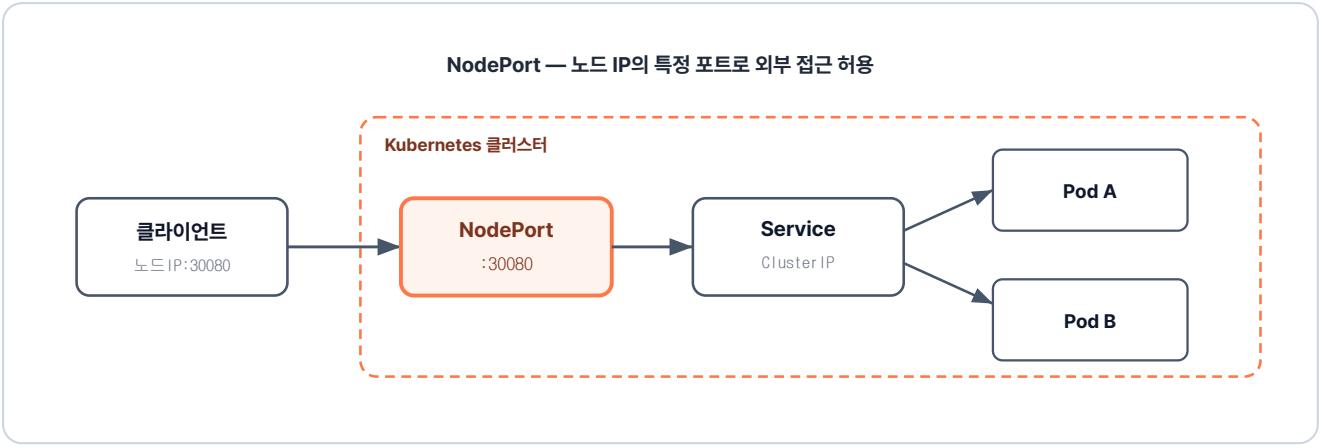


그림 5-6. NodePort는 노드의 특정 포트로 외부 접근을 허용

LoadBalancer

LoadBalancer는 외부 요청을 한곳에서 받아 연결하는 **대표 번호**입니다. Service를 LoadBalancer로 지정하면, AWS·GCP 같은 클라우드 플랫폼이 **공인 IP**를 발급합니다. **그 IP로 들어온 모든 요청을 받아, 뒤에 있는 여러 노드에 분산합니다.** 실제 운영에서 가장 흔히 사용합니다.

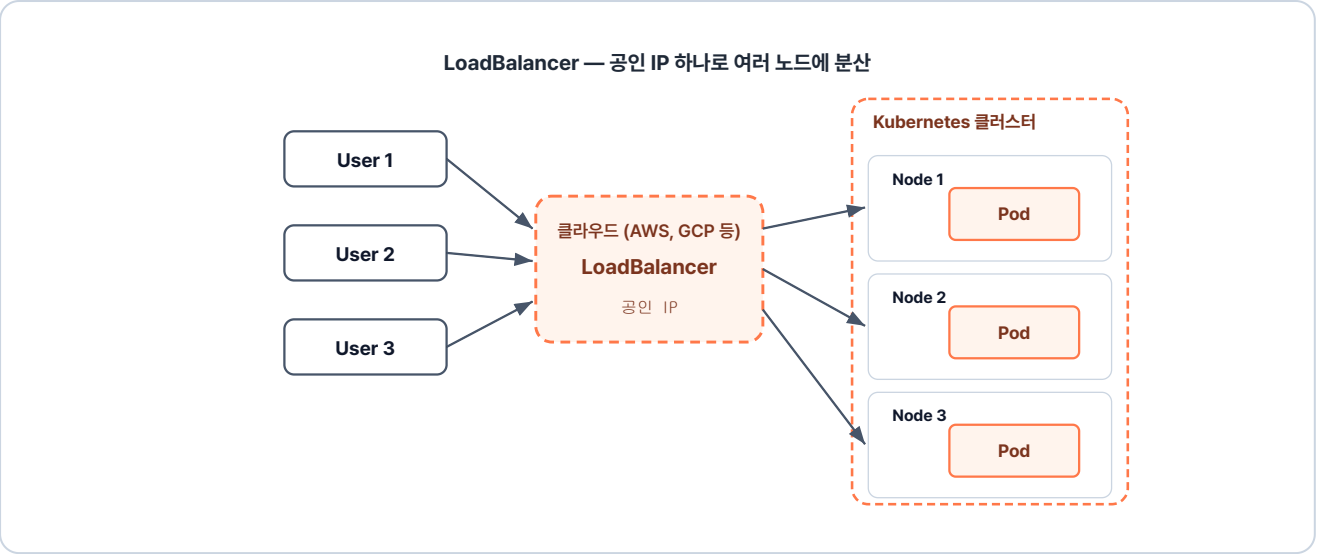


그림 5-7. LoadBalancer는 클라우드가 공인 IP를 발급해 여러 노드에 분산

LoadBalancer 타입은 ClusterIP와 NodePort를 함께 포함합니다. 그래서 Service 하나로 내부 통신과 외부 접근이 모두 처리됩니다.

5.1.4 포트 흐름

Service YAML의 포트 설정에는 `port`, `targetPort`, `nodePort` 세 종류가 있습니다. 각 포트는 외부에서 들어온 요청을 Pod까지 단계별로 전달합니다.

외부 사용자가 노드의 `nodePort`로 접속하면, 요청은 `port`로 열려 있는 Service에 도달하고, Service는 이를 Pod의 `targetPort`로 전달합니다.

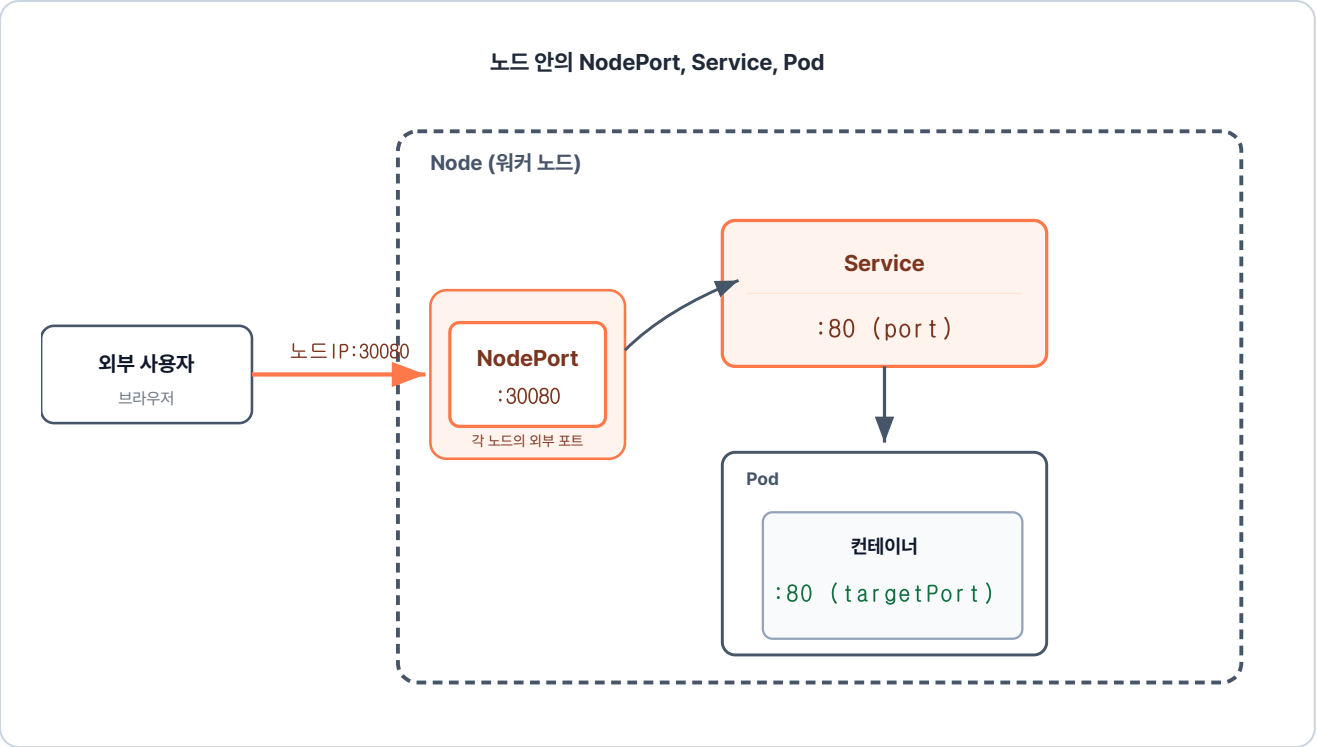


그림 5-8. NodePort에서 Service를 거쳐 Pod 컨테이너까지 이어지는 포트 흐름

포트 종류	위치	역할	생략 시
`nodePort`	워커 노드	외부 사용자가 노드 IP로 클러스터 내부에 접근할 때 열리는 포트	30000~32767 범위에서 자동 할당. 같은 범위로 직접 지정도 가능
`port`	Service	클러스터 내부의 다른 Pod가 이 Service를 호출할 때 쓰는 포트	생략 불가 (필수)
`targetPort`	Pod(컨테이너)	Service가 받은 트래픽을 전달할 컨테이너의 수신 포트 (컨테이너 포트와 일치)	생략 시 `port`와 같은 값

5.1.5 외부에서 Service 접속해 보기

이제 실습을 해 보겠습니다. 아래 명령어를 실행해 클러스터에 적용한 뒤, 외부에서 실제로 접속되는지 확인합니다.

터미널 **Pod와 Service 생성**

```
kubectl apply -f ex11/deploy-ex02.yml    # Pod 4개 생성
kubectl apply -f ex11/service-ex01.yml   # Service YAML 적용
```

실행결과

```
$ kubectl apply -f ex11/deploy-ex02.yml
deployment.apps/nginx-replica created
$ kubectl apply -f ex11/service-ex01.yml
service/nginx-service created
```

그림 5-9. Pod와 Service 생성 결과

브라우저를 열고 NodePort로 설정한 30080 포트(`localhost:30080`)로 접속합니다. 그런데 연결할 수 없다는 화면이 뜹니다.

'분명 30080 포트를 열었는데 왜 안 들어가지.'

> **NodePort로 접속이 안 되는 이유**
> 미니큐브는 내 컴퓨터 안에 작은 가상 환경을 만들고, 그 안에서 클러스터를 실행합니다. 이 가상 환경은 별개의 PC처럼 동작해서, 내 컴퓨터와 다른 네트워크를 가집니다. 그래서 NodePort를 열어도 `localhost` 주소로는 접근할 수 없습니다.

이 문제를 해결하기 위해, 미니큐브에는 내 컴퓨터와 클러스터를 연결하는 별도의 명령어가 있습니다.

방법	명령어	설명
URL 생성	`minikube service <서비스이름> --url`	**Service 한 개에 접근할 임시 URL 생성**. NodePort 접근에 사용
터널 개방	`minikube tunnel`	**LoadBalancer·Ingress에 외부 IP 부여**. LoadBalancer·Ingress 접근에 사용

NodePort에 접근할 때는 임시 URL을 만드는 `minikube service`를 사용합니다. 명령을 실행하면 내 컴퓨터에서 접속할 수 있는 URL이 출력됩니다.

터미널 **Service 접근 URL 생성**

```
minikube service nginx-service --url    # Service 접근 URL 생성
```

실행결과

```
$ minikube service nginx-service --url
http://127.0.0.1:10345
! windows 에서 Docker 드라이버를 사용하고 있기 때문에, 터미널을 열어야 실행할 수 있습니다.
```

그림 5-10. minikube service URL 생성 결과

URL이 출력되고 커서는 그대로 멈춰 있습니다. 이 명령은 연결을 유지하는 동안 터미널에서 계속 실행됩니다. 출력된 주소를 브라우저에 붙여 넣으면 NGINX 환영 페이지가 나타납니다.



그림 5-11. 브라우저에서 nginx 접속 확인

다음으로 Pod가 사라지고 IP가 바뀌어도 Service가 새 Pod로 자동 연결되는지 직접 확인합니다. Ctrl+C로 터미널을 빠져나온 후 Pod를 전부 지웁니다.

터미널 **Pod 재생성 후 같은 주소로 재접속**

```
kubectl delete pod --all                # 모든 Pod 삭제 (Deployment가 자동 재생성)
minikube service nginx-service --url    # 다시 접속 URL 확인
```

실행결과

```
$ kubectl delete pod --all
pod "nginx-replica-756b46b54c-7qztn" deleted
pod "nginx-replica-756b46b54c-cb592" deleted
pod "nginx-replica-756b46b54c-ff5dt" deleted
pod "nginx-replica-756b46b54c-hpnzm" deleted
$ minikube service nginx-service --url
http://127.0.0.1:3082
! windows 에서 Docker 드라이버를 사용하고 있기 때문에, 터미널을 열어야 실행할 수 있습니다.
```

그림 5-12. Pod 삭제 후 Service 접속 확인

새 URL로 접속해도 NGINX 페이지가 그대로 나옵니다.

'Pod 앞에 Service를 두니까, Service로 요청만 보내면 Pod까지 전달되는구나.'

5.2 Ingress - 도메인 라우팅

Service는 Pod 그룹마다 고정 주소를 제공합니다. 하지만 Service는 IP와 포트만으로 요청을 전달할 뿐, 같은 도메인 안에서 **URL 경로에 따라 요청을 나눠 보내지는 못합니다**. 이 한계를 해결하는 리소스가 Ingress입니다.

5.2.1 Ingress의 역할

Ingress는 외부 도메인 하나로 들어온 요청을 **URL 경로에 따라 알맞은 Service로 보냅니다**. 예를 들어 `/order` 경로와 `/stores` 경로를 각각의 Service로 보내도록 규칙을 적어 두면, Ingress가 요청 경로를 읽어 알맞은 Service로 전달합니다.

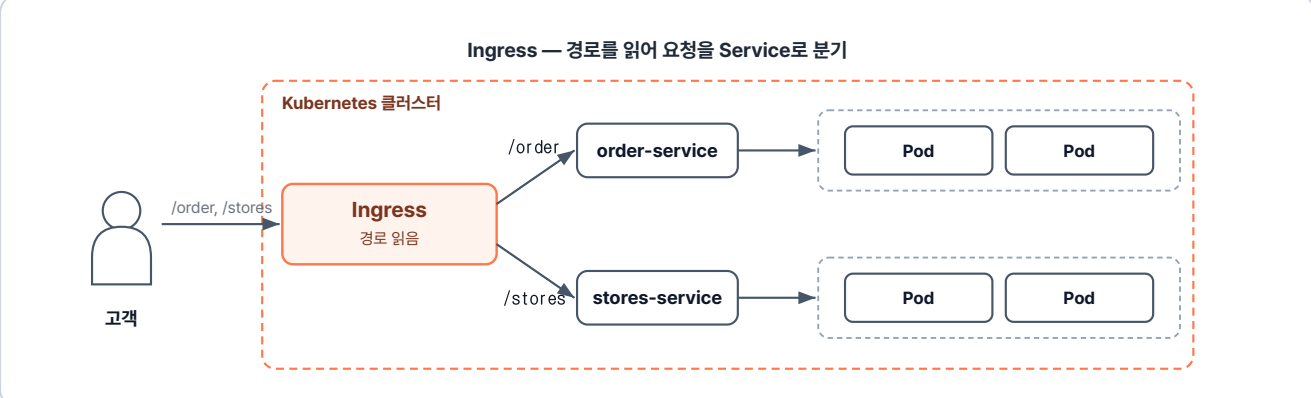


그림 5-13. Ingress가 도메인과 경로를 읽어 요청을 적절한 Service로 연결하는 구조

5.2.2 Ingress 컨트롤러

Ingress는 **Ingress 리소스(YAML)** 와 **Ingress 컨트롤러** 두 가지로 구성됩니다.

Ingress 리소스에는 어떤 경로를 어느 Service로 보낼지 **라우팅 규칙을 적어 둡니다**. 그리고 외부 요청이 들어오면 Ingress 컨트롤러가 **이 규칙대로 알맞은 Service로 전달합니다**.

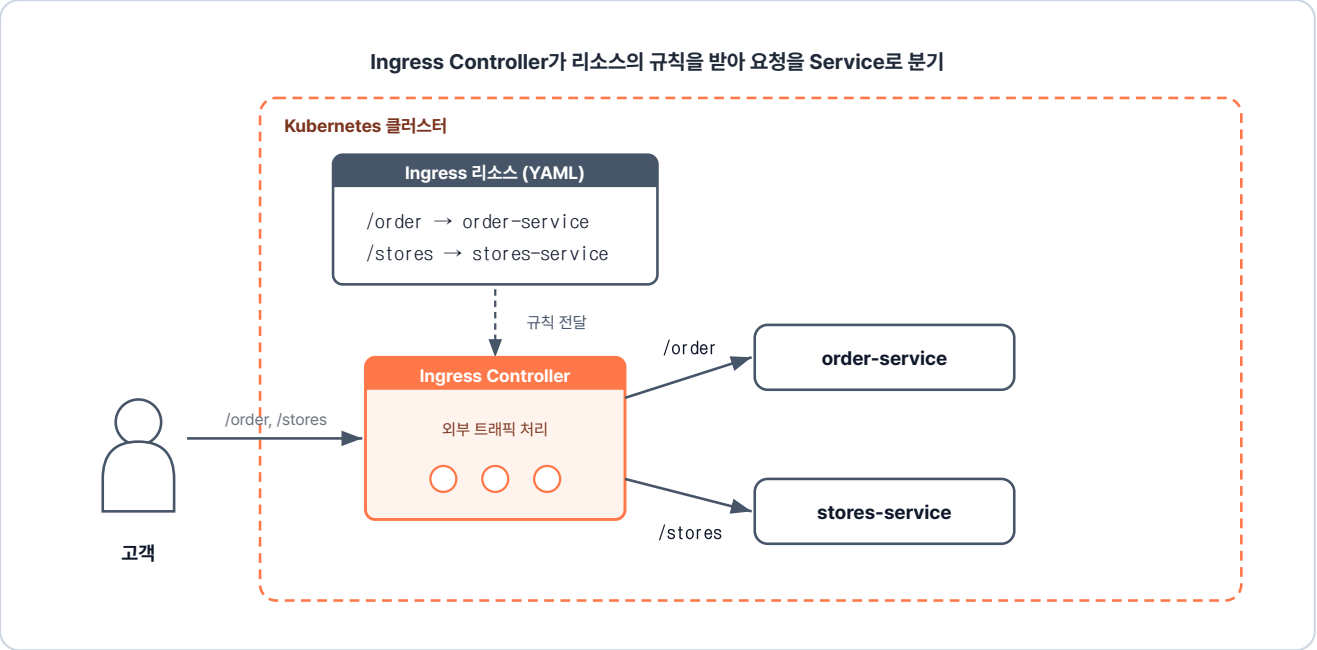


그림 5-14. Ingress 리소스(YAML)와 Ingress 컨트롤러

5.2.3 Ingress 적용하기

실습을 위해 미니큐브에 인그레스를 활성화해 보겠습니다. 미니큐브는 Ingress 컨트롤러를 애드온 형태로 한 번에 배포할 수 있는 명령어를 제공합니다.

터미널 **인그레스 애드온 활성화**

```
minikube addons enable ingress          # 인그레스 애드온 활성화
```

실행 결과

```
$ minikube addons enable ingress
* ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at:
https://github.com/kubernetes/minikube/blob/master/OWNERS
* After the addon is enabled, please run "minikube tunnel" and your ingress resources would
be available at "127.0.0.1"
   - Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
   - Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
   - Using image registry.k8s.io/ingress-nginx/controller:v1.13.2
* Verifying ingress addon...
* The 'ingress' addon is enabled
```

그림 5-15. minikube에서 ingress 애드온 활성화 과정

Ingress 컨트롤러가 실제로 떠 있는지 확인합니다.

터미널 **Ingress 컨트롤러 확인**

```
kubectl get pods -n ingress-nginx        # Ingress 컨트롤러 확인
```

실행 결과

```
$ kubectl get pods -n ingress-nginx
NAME READY STATUS RESTARTS AGE
ingress-nginx-admission-create-zg9hh 0/1 Completed 0 40s
ingress-nginx-admission-patch-bvxlw 0/1 Completed 1 40s
ingress-nginx-controller-9cc49f96f-qrpm7 1/1 Running 0 35s
```

그림 5-16. Ingress 컨트롤러가 Running 상태임을 확인

실제 환경에서는 어떻게 띄울까

실제 서비스 환경에서는 Ingress 컨트롤러를 클러스터에 직접 설치해야 합니다. 그런 다음 외부 사용자가 접속할 수 있도록 클라우드의 공인 IP와 연결하는 작업이 필요합니다. 앞서 사용한 미니큐브 애드온은 이 두 단계를 명령어 하나로 처리해 줍니다.

`ex12` 폴더에는 Ingress 규칙과 두 Service, 그리고 각 Service가 가리키는 Deployment가 함께 들어 있습니다.

```
ex12/
├── ingress-ex01.yml   # /order·/stores 경로별 라우팅 규칙
├── order-deploy.yml    # 주문 응답 Pod Deployment
├── order-service.yml   # 주문 Pod 앞 ClusterIP Service
├── stores-deploy.yml   # 매장 응답 Pod Deployment
├── stores-service.yml  # 매장 Pod 앞 ClusterIP Service
└── README.md           # 실습 안내
```

이제 외부 요청을 주문 Service와 매장 Service로 나눠 보낼 **Ingress 규칙**을 만들 차례입니다. 두 Service는 **ClusterIP** 타입이라 클러스터 바깥에서는 직접 접근할 수 없습니다.

다음은 `/order`와 `/stores` 두 경로를 각각의 Service로 분기하는 Ingress YAML입니다.

실습 2 **ex12/ingress-ex01.yml. 경로별 라우팅 규칙**

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ex12-ingress
spec:
  ingressClassName: nginx           # 어느 Ingress 컨트롤러가 이 규칙을 집행할지 지정
  rules:
    - http:
        paths:
          - path: /order            # 주문 경로 → order-service
            pathType: Prefix        # /order로 시작하는 하위 경로까지 매칭
            backend:
              service:
                name: order-service
                port:
                  number: 5678
          - path: /stores           # 매장 경로 → stores-service
            pathType: Prefix        # /stores로 시작하는 하위 경로까지 매칭
            backend:
              service:
                name: stores-service
                port:
                  number: 5678
```

이제 `ex12` 폴더의 모든 파일을 한 번에 적용합니다.

터미널 **Ingress 규칙과 두 Service 일괄 적용**

```
kubectl apply -f ex12/                    # ex12 폴더 내 모든 yaml 일괄 적용
```

실행결과

```
$ kubectl apply -f ex12/
ingress.networking.k8s.io/ex12-ingress created
deployment.apps/order-deploy created
service/order-service created
deployment.apps/stores-deploy created
service/stores-service created
```

그림 5-17. Ingress 리소스 등록 확인

5.2.4 외부에서 Ingress 접속해 보기

클러스터를 호스트와 연결하기 위해, 외부 IP를 부여하는 `minikube tunnel`을 별도 터미널에서 실행합니다.

터미널 **외부 터널 개방**

```
minikube tunnel                           # 별도 터미널에서 실행
```

실행결과

```
$ minikube tunnel
* Tunnel successfully started
* NOTE: Please do not close this terminal as this process must stay alive for the tunnel to be accessible ...
* Starting tunnel for service ex12-ingress.
```

그림 5-18. minikube tunnel 실행 화면

이제 브라우저에서 두 경로를 차례로 접속해 보겠습니다.

먼저 `http://localhost/order`를 열면 "주문 접수 완료"가 나옵니다.

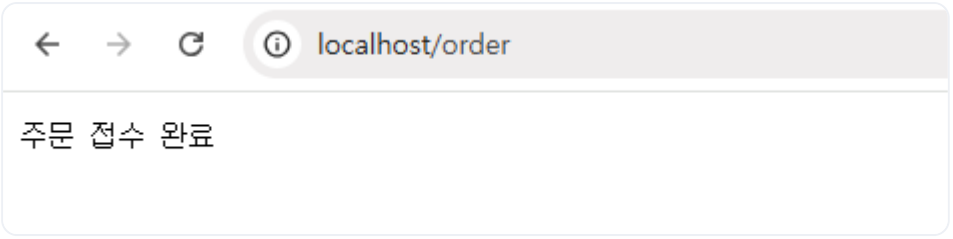


그림 5-19. /order 접속 결과

이어서 `http://localhost/stores`를 열면 "매장 선택"이 나옵니다.

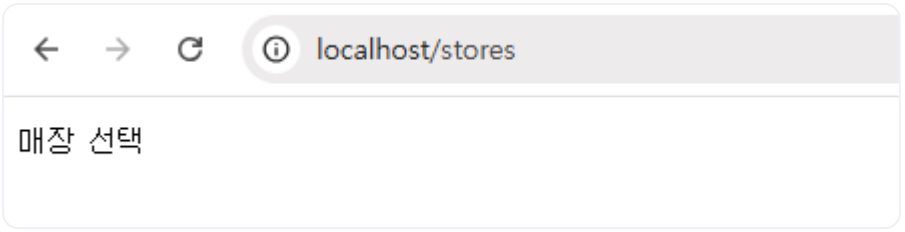


그림 5-20. /stores 접속 결과

도메인은 하나인데 경로마다 다른 응답이 돌아왔습니다. Ingress가 경로를 보고 알맞은 **Service**로 요청을 넘기고, Service가 다시 그 뒤의 **Pod**로 요청을 전달해 응답이 만들어진 결과입니다.

그럼 한 요청 안에서 Ingress와 Service는 각각 어떤 정보를 보고 처리하는 걸까?

5.2.5 두 리소스의 계층 분담

두 리소스가 같은 요청에서 보는 정보가 다른 모습은 우편 봉투의 송장과 닮았습니다.

우편 송장에는 두 가지 정보가 함께 적혀 있습니다. 물류 센터가 어느 지역으로 보낼지 정할 때 찍는 **송장 바코드**, 그리고 그 지역 우체국에서 **집배원**이 직접 눈으로 확인하는 **받는 곳 주소**입니다.



그림 5-21. 우편 위 송장에 바코드와 받는 곳 주소가 함께 있고, 물류 센터는 바코드까지, 집배원은 받는 곳 주소만 읽습니다

이 우편 송장의 구조를 네트워크 **패킷**에 그대로 옮겨 보겠습니다. 패킷은 **클라이언트와 서버가 네트워크로 데이터를 주고받는 기본 단위**입니다.

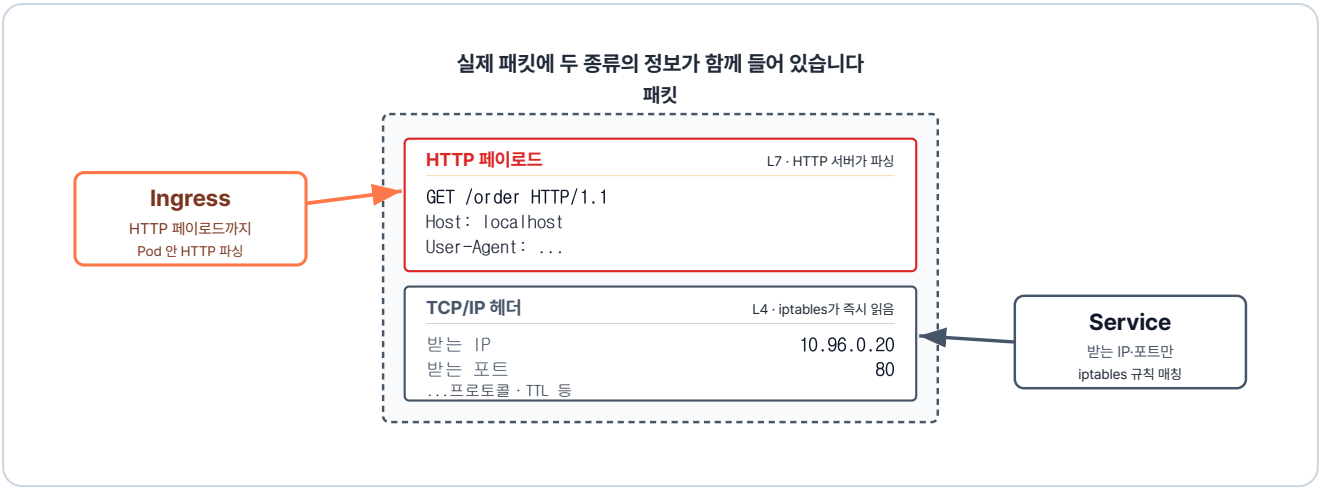


그림 5-22. 실제 패킷에 HTTP 페이로드와 TCP/IP 헤더가 함께 들어 있고, Ingress는 페이로드까지, Service는 헤더만 읽습니다

패킷 안에는 우편 송장처럼 역할이 다른 여러 종류의 데이터가 섞여 있습니다. 이 정보들은 각자의 역할에 따라 **전송 계층**(Layer 4)과 **응용 계층**(Layer 7)에서 각각 처리됩니다.

구분	전송 계층 (L4)	응용 계층 (L7)
역할	데이터를 어느 컴퓨터의 어느 프로세스로 보낼지 안내	패킷 내부의 메시지·콘텐츠를 해석
보는 정보	패킷 겉면(헤더)의 받는 IP·포트	패킷 내부의 URL·도메인
담당 리소스	**Service**	**Ingress**
동작	지정된 IP·포트로 들어온 요청을 백엔드 Pod로 분산·전달	세부 경로(`/order`, `/stores`)에 따라 알맞은 Service로 전달

5.3 브라우저에서 Pod까지의 경로

이전 챕터에서는 개발자의 명령으로 Pod를 띄우고 관리하는 흐름을 다뤘습니다. 이번에는 사용자가 브라우저에서 요청했을 때 Pod에 도달하는 흐름을, 우체국에 우편을 보내는 예시를 통해 단계별로 알아보겠습니다.

5.3.1 NodePort로 클러스터에 들어오기

먼저 발신자가 **중앙 우체국** 창구에 우편을 맡깁니다. 우편을 받은 **창구 직원**은 다음 거점인 **물류 센터**로 보냅니다.

브라우저가 보낸 요청도 비슷하게 흘러갑니다. 요청이 노드에 열린 **NodePort**로 클러스터에 들어오면, 같은 노드의 **kube-proxy**가 패킷의 목적지를 다음 거점인 **Ingress 컨트롤러**의 IP로 바꿉니다. 그러면 패킷은 Ingress 컨트롤러로 전달됩니다.

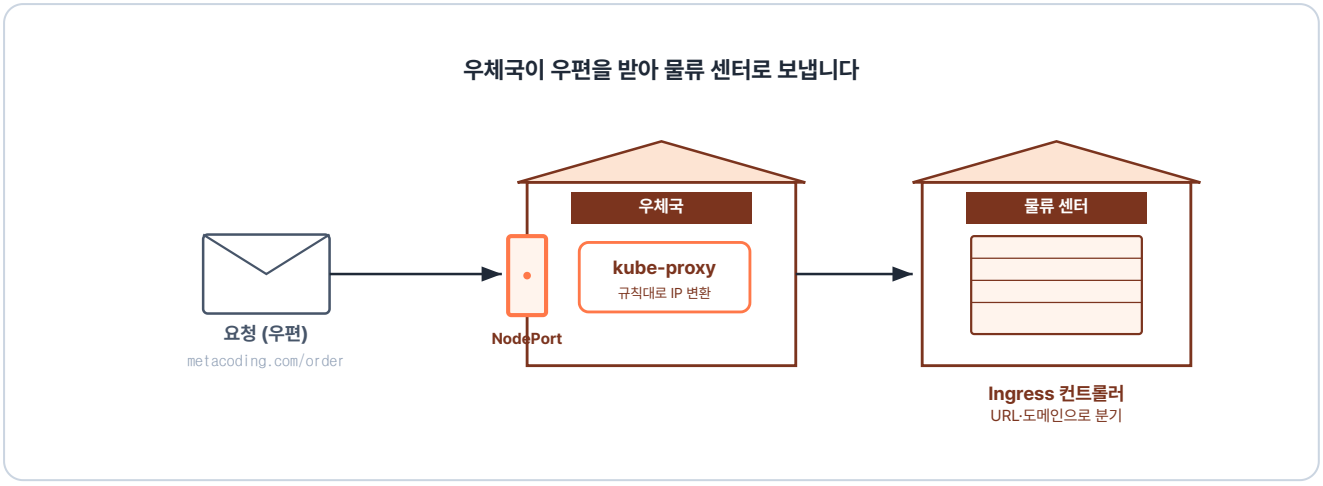


그림 5-23. 요청이 NodePort로 들어와 Ingress 컨트롤러로 향합니다

컴포넌트	이 단계에서 하는 일
NodePort	노드에 포트를 열어 외부 요청을 클러스터 안으로 받는 입구
kube-proxy	패킷의 목적지를 노드 IP에서 Ingress 컨트롤러의 IP로 바꿈

5.3.2 Ingress가 URL·도메인으로 Service 분기하기

우편이 **물류 센터**에 도착합니다. 물류 센터에서는 바코드를 스캐너로 찍어 등록된 **분류 규칙**에 따라 어느 지역 우체국으로 갈지 정합니다.

Ingress 컨트롤러도 **Ingress 리소스**에 적힌 규칙으로 요청을 보낼 곳을 정합니다. 요청의 **URL 경로와 도메인**을 그 규칙과 비교해 맞는 **Service**를 고릅니다.

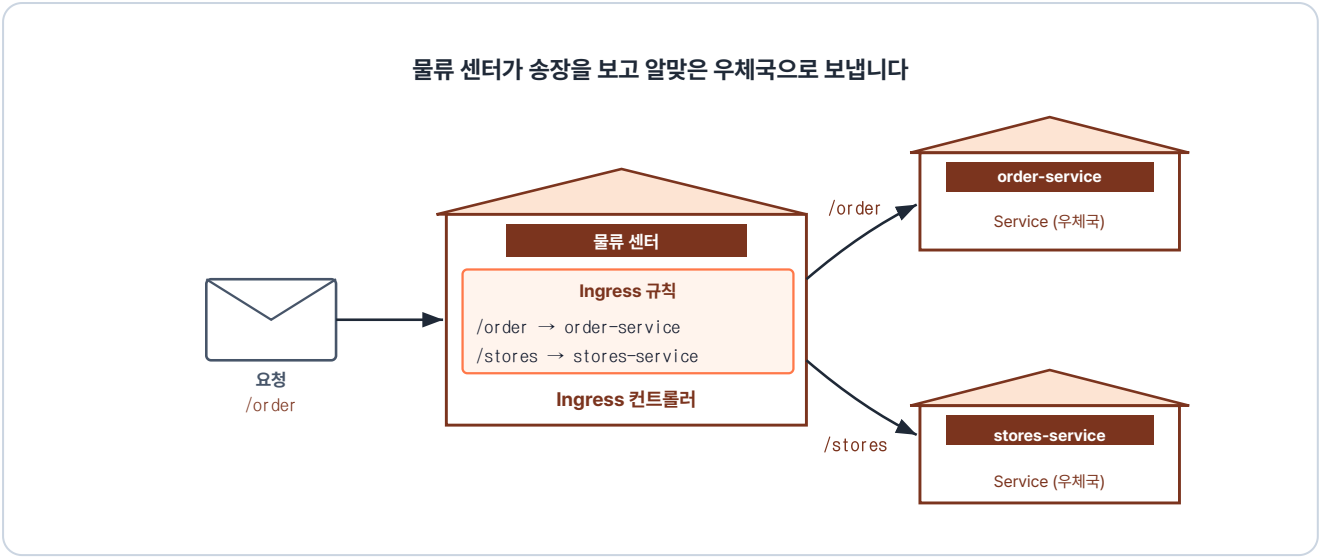


그림 5-24. Ingress 컨트롤러가 경로에 따라 알맞은 Service로 보냅니다

컴포넌트	이 단계에서 하는 일
Ingress 컨트롤러	요청을 받아 URL 경로·도메인에 맞는 Service로 전달
Ingress 규칙	어떤 경로·도메인을 어느 Service로 보낼지 적어 둔 설정

5.3.3 ClusterIP가 Pod로 전달하기

물류 센터에서 우체국으로 도착한 우편은 이제 배송이 시작됩니다. **집배원**은 **배송 목록**에 적힌 **우편함**에 우편을 넣습니다.

마지막 단계는 **kube-proxy**가 담당합니다. kube-proxy는 도착한 패킷의 IP를 **EndpointSlice**에서 살아있는 **Pod** 중 하나의 IP로 바꿉니다. 그러면 패킷은 그 Pod로 전달됩니다.

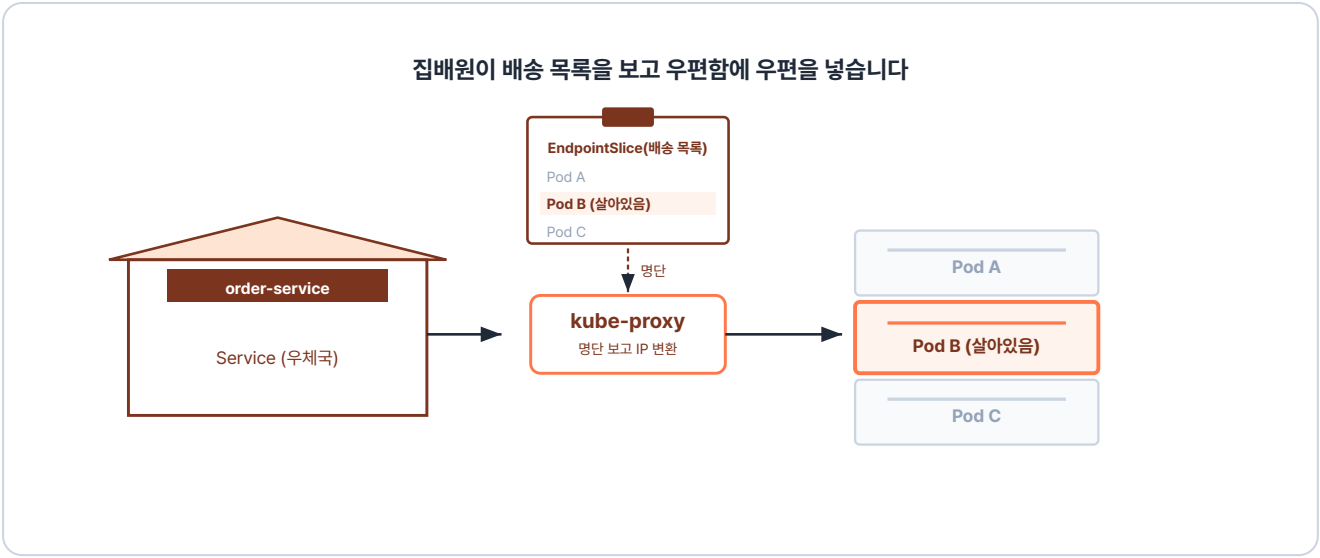


그림 5-25. kube-proxy가 살아있는 Pod로 목적지를 바꿉니다

컴포넌트	이 단계에서 하는 일
Service (ClusterIP)	여러 Pod를 대표하는 고정 IP로, 요청이 이 주소로 들어옴
EndpointSlice	Service에 연결된, 지금 살아있는 Pod들의 IP 목록
kube-proxy	요청의 목적지를 ClusterIP에서 살아있는 Pod의 IP로 바꿈

Ingress 컨트롤러와 kube-proxy가 설정을 받는 방법

클러스터에 등록된 모든 설정은 **API Server**를 거쳐 상태 저장소인 **etcd**에 보관됩니다. 각 컴포넌트는 API Server로부터 자신에게 필요한 설정을 전달받아 적용합니다. 이때 Ingress 컨트롤러는 YAML로 작성된 **Ingress 리소스**를 받아 라우팅 규칙으로 사용하고, kube-proxy는 살아있는 Pod 목록인 **EndpointSlice**를 받아 IP 변환에 반영합니다.

이번 챕터에서는 외부에서 들어온 요청이 클러스터 안의 Pod까지 도달하는 흐름을 살펴봤습니다. 하나의 요청이 들어오면 Ingress는 도메인과 URL 경로를, Service는 IP와 포트를 확인해 목적지를 결정합니다.

'이제 요청이 클러스터 안에서 Pod까지 닿는 구조를 알았으니, 내 프로젝트에도 직접 적용해 볼 수 있겠다.'

다음 챕터에서는 실제 운영 환경에 필요한 환경 변수와 데이터 저장소를 다뤄 보겠습니다.

이것만은 기억하자

- **Service는 Pod의 고정 진입점입니다.** Pod는 소모품이라 IP가 수시로 바뀌지만, Service는 변하지 않는 주소를 제공하며 트래픽을 분산합니다.
- **Service는 접근 범위에 따라 세 종류로 나뉩니다. ClusterIP**는 클러스터 안에서만 통신합니다. **NodePort**는 노드의 IP와 포트로 외부에 열립니다. **LoadBalancer**는 클라우드가 발급한 공인 IP로 외부에 열립니다.
- **Ingress는 외부 요청을 URL 경로로 분기합니다.** 숫자(IP·포트)만 보는 Service와 달리, 도메인과 URL 경로를 읽어 알맞은 Service로 연결합니다. 라우팅 규칙을 정의하는 **리소스(YAML)** 와 실제 요청을 처리하는 **Ingress 컨트롤러(Pod)** 가 함께 동작합니다.